

操作系统作业3

202440012028 陈新安

1、现有3个优先级为P1>P2>P3的抢占式调度进程，共享临界资源S用二元信号量保护。已知P3先持有S进入临界区，随后P2就绪抢占P3的CPU，之后P1就绪请求S被阻塞。请回答:① 该场景出现了什么同步问题?② 分别用优先级继承协议、优先级天花板协议写出完整的信号量操作逻辑，对比两种方案下的进程调度序列差异。

出现了优先级反转的问题，因为P3持有资源S但被P2抢占，导致P1无法获取资源S而被阻塞。P3的优先级被P2抢占后降低，无法及时释放资源S给P1。

```
Semaphore S = 1;
...
void Task_P3() {
    P(S);          // P3 最先运行，成功获取资源 S
    CriticalSection();
    // 此处被 P2 抢占，无法执行 V(S)
    V(S);          // 释放资源 S
}
```

优先级继承协议：当P2抢占P3时，P3暂时继承P2的优先级，直到释放资源S。这样P3可以继续执行并释放S给P1。
题中的例子：

- 1. P3 获取信号量 S 。此时 P3 仍以原优先级运行。
- 2. P2 就绪。因为 $P2 > P3$ ，**P2 抢占 P3**，P3 带着锁进入就绪队列。
- 3. P1 就绪并抢占 P2，随后 P1 请求信号量 S 。发现 S 被 P3 占用，**P1 阻塞**。
- 4. 内核检测到 P1（高）被 P3（低）阻塞，立即将 **P3 的优先级提升至 P1 的水平**。
- 5. 由于此时 $P3 > P2$ ，**P3 抢占 P2** 恢复执行，直到释放 S 。
- 6. P3 执行 $v(s)$ 释放资源。内核立即**将 P3 的优先级还原**。此时 P1 不再受阻，抢占 P3 和 P2 执行。

优先级天花板协议：在此协议下，资源S被赋予一个天花板优先级（通常是系统中最高的优先级）。当P3获取S时，它的优先级立即提升到天花板优先级，防止被P2抢占。这样P1可以直接获取S，无需等待P3释放。
题中的例子：

- 1. 信号量 S 的天花板优先级设为系统最高优先级（例如 P1）。
- 2. P3 获取信号量 S 。根据天花板协议，**P3 的优先级立即提升至 P1 的水平**。
- 3. P2 就绪。由于 P3 已提升到 P1 的优先级，**P2 无法抢占 P3**，P3 继续执行。
- 4. P1 就绪并请求信号量 S 。由于 P3 已提升到 P1 的优先级，**P1 无法抢占 P3**，P1 阻塞。
- 5. P3 执行 $v(s)$ 释放资源。内核立即**将 P3 的优先级还原**。此时 P1 不再受阻，抢占 P3 和 P2 执行。

2、多生产者多消费者变种:三类生产者A/B/C分别生产a/b/c三类产品，两类消费者X需要消费a+b组合、Y需要消费b+c组合，环形缓冲区最多存放10个任意产品，额外要求缓冲区中a的数量不能比b多3个、b的数量不能比c多2个。请用信号量实现同步互斥，定义所有信号量的含义与初值，证明你的实现不会死锁或给出死锁触发条件。

初始化资源 定义以下信号量对象：

```
int in = 0, out = 0;
item buffer[10];
semaphore mutex = 1, empty = 10, full = 0;
semaphore sa = 0, sb = 0, sc = 0; // 分别表示a、b、c的数量
semaphore L_ab = 3, L_bc = 2; // 分别表示a比b多3个、b比c多2个的限制
```

使用 AND 型信号量实现生产者和消费者的同步互斥：

生产者 A	生产者 B	生产者 C
<pre>void ProducerA() { do { item nextp_a = produce_a(); Swait(empty, L_ab, mutex); buffer[in] = nextp_a; in = (in + 1) % 10; Ssignal(mutex, sa, full); } while {TRUE}; }</pre>	<pre>void ProducerB() { do { item nextp_b = produce_b(); Swait(empty, L_bc, mutex); buffer[in] = nextp_b; in = (in + 1) % 10; Ssignal(mutex, sb, full, L_ab); } while {TRUE}; }</pre>	<pre>void ProducerC() { do { item nextp_c = produce_c(); Swait(empty, mutex); buffer[in] = nextp_c; in = (in + 1) % 10; Ssignal(mutex, sc, full, L_bc); } while {TRUE}; }</pre>

消费者 X	消费者 Y
<pre>void ConsumerX() { while (true) { Swait(sa, sb, full, mutex); nextp_a = buffer[out]; out = (out + 1) % 10; nextp_b = buffer[out]; out = (out + 1) % 10; Ssignal(mutex, empty, empty, L_ab); consume_a(nextp_a); consume_b(nextp_b); } }</pre>	<pre>void ConsumerY() { while (true) { Swait(sb, sc, full, mutex); nextp_b = buffer[out]; out = (out + 1) % 10; nextp_c = buffer[out]; out = (out + 1) % 10; Ssignal(mutex, empty, empty, L_bc); consume_b(nextp_b); consume_c(nextp_c); } }</pre>

死锁分析：会出现死锁，产生死锁的条件之一如下：
因为消费者Y需要同时等待sb和sc信号量。当缓冲区内全都是c产品时，sb信号量为0，消费者Y被阻塞；生产者b想生产b产品，但由于empty信号量为0（缓冲区满），生产者b也被阻塞；生产者c想生产c产品，但由于empty信号量为0，也被阻塞。此时消费者Y、生产者b和生产者c形成了一个循环等待的死锁状态。

3、分层读者写者问题:读者分为A类（无并发数限制）、B类（最多同时2个并发），写者与所有读者互斥，且写者绝对优先:只要有写者等待，后续新来的任何读者都不允许进入临界区。请用信号量实现该逻辑，分析该机制的读写公平性，以及临界区最多同时存在多少个进程。

信号量定义：

```
semaphore mutex_A = 1, mutex_B = 1, mutex_W = 1; // 保护读者计数和写者计数的互斥锁
semaphore rw_mutex = 1; // 保护临界区的读写互斥锁
semaphore W_wait = 1; // 写者等待信号量，控制写者优先
semaphore B_limit = 2; // 限制B类读者的并发
int count_A = 0, count_B = 0, count_W = 0;
```

写者

```
void Writer() {
    P(mutex_W);
    count_W++;
    if (count_W == 1) P(W_wait); // 信号量中维护的阻塞队列保证优先性
    V(mutex_W);

    P(rw_mutex);    // 申请进入临界区写操作
    // [正在写...]
    V(rw_mutex);    // 释放临界区

    P(mutex_W);
    count_W--;
    if (count_W == 0) V(W_wait); // 直到所有写者（包括排队的）都走光，才给读者开门
    V(mutex_W);
}
```

A类读者	B类读者
------	------

<pre>void ReaderA() { P(W_wait); P(mutex_A); count_A++; if (count_A == 1 && count_B == 0) P(rw_mutex); V(mutex_A); V(W_wait); // [正在读...] P(mutex_A); count_A--; if (count_A == 0 && count_B == 0) V(rw_mutex); V(mutex_A); }</pre>	<pre>void ReaderB() { P(W_wait); P(B_limit); P(mutex_B); count_B++; if (count_B == 1 && count_A == 0) P(rw_mutex); V(mutex_B); V(W_wait); // [正在读...] P(mutex_B); count_B--; if (count_A == 0 && count_B == 0) V(rw_mutex); V(mutex_B); V(B_limit); }</pre>
---	---

读写公平性：极度不公平，写者绝对优先。

临界区最大并发数：当没有写者时，A类读者数量无限制，B类读者最多2个，因此最大并发数为无限制的A类读者 + 2个B类读者 = 无限大；当有写者时，写者独占临界区，最大并发数为1。

实际上，受制于硬件限制，假设最多有N个A类读者，那么临界区最大并发数为N + 2（当没有写者时）。当有写者时，最大并发数为1。

4、优先级哲学家进餐问题:5个哲学家围坐，共有3双共6根筷子，每个哲学家就餐需要持有2根筷子；哲学家分为饥饿优先级、普通优先级两类，饥饿哲学家拿筷子的优先级高于普通哲学家。请用信号量实现，要求无死锁、无普通哲学家饿死，计算筷子资源的最大利用率。

信号量定义：

```
semaphore chopsticks = 3; // 3双筷子
semaphore mutex_h = 1; // 保护 hungry_waiting 计数
int hungry_waiting = 0; // 队列中的饥饿哲学家数量
semaphore hunger_gate = 1; // 优先级门控
semaphore mutex = 1; // 互斥锁
```

饥饿的哲学家	普通的哲学家
<pre>void Hungry_Philosopher() { while (true) { P(mutex_h); hungry_waiting++; if (hungry_waiting == 1) { P(hunger_gate); } V(mutex_h); P(mutex); P(chopsticks); // 获取一双筷子 V(mutex); // [就餐中...] V(chopsticks); P(mutex_h); hungry_waiting--; if (hungry_waiting == 0) { // 所有饥饿者吃完 V(hunger_gate); } V(mutex_h); } }</pre>	<pre>void Normal_Philosopher() { while (true) { P(hunger_gate); V(hunger_gate); P(mutex); P(chopsticks); V(mutex); // [就餐中...] V(chopsticks); } }</pre>

最大利用率：只要总人数不少于3人，筷子资源就能被完全利用（3双筷子同时被3个人使用）。因此最大利用率为100%。

5、多核信号量实现:二元信号量的P操作依赖原子Test-and-Set指令实现，若没有添加内存屏障，在双核CPU上会出现什么逻辑错误?请写出带内存屏障的P/V操作实现，分析内存屏障带来的性能开销。

在多核CPU上，如果二元信号量的P操作没有添加内存屏障，可能会出现以下逻辑错误：

- 1. **指令重排**：编译器或CPU可能会对指令进行重排优化，导致信号量状态的更新和临界区代码的执行顺序发生变化。这可能导致一个线程在进入临界区之前就释放了信号量，另一个线程可能会错误地认为信号量已经被释放，从而进入临界区，造成数据不一致或竞争条件。
- 2. **缓存一致性问题**：在多核系统中，每个核心可能有自己的缓存，如果没有内存屏障，某个核心对信号量状态的修改可能不会立即被其他核心看到。这可能导致多个线程同时认为信号量可用，从而进入临界区，造成数据损坏。

带内存屏障的P/V操作实现：

```
void P(semaphore s) {
    while (TestAndSet(&s.guard));

    // 保证后续对 s.value 的读取不被重排到 TAS 之前
    Memory_Barrier_Acquire();

    s.value--;

    if (s.value < 0) {
        放入等待队列 s.queue;

        // 保证放入队列的操作对其他核可见
        Memory_Barrier_Release();

        s.guard = 0; // 释放自旋锁
        p.sleep(P); // 阻塞当前线程
    } else {

        Memory_Barrier_Release();

        s.guard = 0; // 释放自旋锁
    }
}

void V(semaphore s) {
    // 1. 获取自旋锁
    while (TestAndSet(&s.guard));

    //
    Memory_Barrier_Acquire();

    s.value++;

    if (s.value <= 0) {
        //从等待队列 s.queue 取出一个进程 P;
        p.wake(P); // 唤醒进程 P
    }

    //保证 value 修改和唤醒准备工作已完成
    Memory_Barrier_Release();

    s.guard = 0; // 释放自旋锁
}
```

性能开销分析：内存屏障会引入一定的性能开销，因为它会阻止指令重排，确保内存操作的顺序性。这可能导致CPU无法进行某些优化，例如指令级并行和缓存优化，从而增加了P/V操作的执行时间。此外，在高竞争的情况下，频繁的内存屏障可能会导致更多的缓存一致性流量，进一步降低性能。然而，为了保证多核系统中的正确性，这些开销是必要的。

6、复杂公交同步问题:公交车有前门（仅上客）、后门（仅下客），同一时间只能开一个门，车辆核载40人；站台乘客分为上车、下车两类，上车乘客需要刷NFC计费，计费系统最多支持3个乘客同时刷卡，下车乘客无需刷卡；司机停稳后才能开门，所有乘客上下完成后按铃通知司机，司机收到通知才能关门开车。请用信号量实现司机、上车乘客、下车乘客的同步互斥，分析高峰期的资源利用率。

信号量定义：

```
semaphore bus_capacity = 40;
semaphore card_machine = 3;
semaphore door_mutex = 1;

semaphore stop_stable = 0;
semaphore finish_up = 0;
semaphore finish_down = 0;
semaphore bell = 0;

int count_up = 0;
semaphore mutex_up = 1;
int count_down = 0;
semaphore mutex_down = 1;
```

司机：

```
void Driver() {
    while (true) {
        // 到站，停稳
        P(stop_stable);

        // 开门
        P(door_mutex);
        OpenDoor();
        V(door_mutex);

        // 等待乘客上下分别完成
        P(bell); P(bell);

        // 关门开车
        P(door_mutex);
        CloseDoor();
        V(door_mutex);
    }
}
```

上车乘客：

```
void BoardingPassenger() {
    while (true) {
        P(bus_capacity); // 等待车内有空位

        P(card_machine); // 等待刷卡机可用
        SwipeCard();      // 刷卡计费
        V(card_machine); // 释放刷卡机

        P(mutex_up);
        count_up++;
        if (count_up == 1) {
            P(finish_down); // 如果是第一个上车的乘客，等待下车乘客完成
        }
        V(mutex_up);

        // [上车中...]

        P(mutex_up);
        count_up--;
        if (count_up == 0) {
            V(finish_up); // 最后一个上车的乘客通知司机
            V(bell);
        }
        V(mutex_up);
    }
}
```

下车乘客：

```

void AlightingPassenger() {
    while (true) {
        // [下车中...]

        P(mutex_down);
        count_down++;
        if (count_down == 1) {
            P(finish_up); // 如果是第一个下车的乘客，等待上车乘客完成
        }
        V(mutex_down);

        V(bus_capacity); // 离开车内，释放一个座位

        P(mutex_down);
        count_down--;
        if (count_down == 0) {
            V(finish_down); // 最后一个下车的乘客通知司机
            V(bell);
        }
        V(mutex_down);
    }
}

```

高峰期资源利用率：上车和下车是串行的，谁先执行取决于是上车乘客还是下车乘客先到达。无论哪种情况，资源利用率都可以达到100%，因为司机、上车乘客和下车乘客之间的同步机制确保了每个阶段的操作都能顺利进行，不会出现资源闲置的情况。

7、VIP睡眠理发师问题:理发店有3个理发师、10个等待座位，顾客分为VIP和普通两类，VIP顾客无空理发师时可直接插队到等待队列最前，普通顾客按先来先服务排队；理发师每次服务完主动叫下一个顾客，VIP优先。请用信号量实现，要求不会出现普通顾客饿死，给出普通顾客等待时间的上下界。

由于信号量的局限性，一旦某个进程没有分配到资源，进入等待队列后就无法再改变位置，因此无法实现VIP顾客直接插队到等待队列最前的功能。由此，可以维护一个额外的VIP等待队列和一个普通等待队列。理发师优先从VIP队列中叫顾客，如果VIP队列为空再从普通队列中叫顾客。这样子实现了VIP优先的同时，普通顾客也不会饿死。缺点是资源的开销更大了，且普通用户的等待时间会增加。

信号量定义：

```

semaphore seats = 10;
semaphore barbers = 3;
semaphore mutex = 1;

semaphore vip_pool = 0;
semaphore norm_pool = 0;

// 计数器
int vip_wait = 0;
int norm_wait = 0;
int vip_served_count = 0; // 防止普通客户饿死
const int MAX_VIP_STREAK = 5; // 防止普通客户饿死

```

理发师：

```
void Barber() {
    while (true) {
        P(mutex);

        bool serve_vip = false;
        if (vip_wait > 0) {
            if (norm_wait == 0 || vip_served_count < MAX_VIP_STREAK) {
                serve_vip = true; // 有 VIP 且没达到阈值, 或没普通人, 就服务 VIP
            }
        }

        if (serve_vip) {
            vip_wait--;
            vip_served_count++;
            V(vip_pool);    // 唤醒 VIP 队列
        } else if (norm_wait > 0) {
            norm_wait--;
            vip_served_count = 0;
            V(norm_pool);   // 唤醒普通队列
        } else {
            // 没人, 理发师什么都不用做, 等待就好了
        }

        V(mutex);

        // [理发]
    }
}
```

VIP顾客	普通顾客
<pre>void VIP_Customer() { P(mutex); if (vip_wait + norm_wait < 10) { // 有空位 vip_wait++; V(mutex); P(vip_pool); // [被唤醒后, 剪发中...] } else { V(mutex); // 没位子, 走人 } }</pre>	<pre>void Normal_Customer() { P(mutex); if (vip_wait + norm_wait < 10) { norm_wait++; V(mutex); P(norm_pool); // 在普通等候区阻塞 // [被唤醒后, 剪发中...] } else { V(mutex); } }</pre>

普通用户等待时间下界：0

普通用户等待时间上界：假设当前顾客是一个普通顾客，且他在等待队列的最后一个，前面九个顾客都普通顾客。每当一个普通顾客理完发，接下来紧接着就有一个VIP顾客到来并插队，导致这个普通顾客一直被推迟。由于设定了单个理发师的VIP服务连续次数上限为MAX_VIP_STREAK，因此在最坏情况下，每理发完一个普通顾客后，最多会有MAX_VIP_STREAK个VIP顾客插队。假设每个理发师理发时间为T，那么这个普通顾客的等待时间上界为： $9 \times T + 3 \times MAX_VIP_STREAK \times T$

本解法中，MAX_VIP_STREAK设为5，故普通顾客等待时间上界为：24 T，

8、可变大小缓冲区生产者消费者:缓冲区为链式无界结构，但系统内存最多允许存储100个链表节点；生产者生产的产品大小为1~3个节点不等，消费者每次消费1个产品并释放对应数量的节点。请用信号量实现同步互斥，要求不需要遍历链表计算剩余空间，分析你设计的信号量的空间开销。

信号量定义：

```
semaphore mutex = 1;           // 保护链表物理操作的互斥锁
semaphore items = 0;           // 缓冲区中“产品”的数量（注意：不是节点数）
semaphore memory_cells = 100; // 剩余可用的“内存节点”数量
```

生产者	消费者
-----	-----

```

void Producer() {
    while (true) {
        int size = random(1, 3);

        for (int i = 0; i < size; i++) {
            P(memory_cells);

        }

        P(mutex);
        // [创建并插入 size 个节点到链表尾部]
        V(mutex);

        //增加产品计数
        V(items);
    }
}

```

```

void Consumer() {
    while (true) {
        P(items);

        P(mutex);
        // 取出一个产品
        int size = get_head_product_size();
        // [移除产品并释放节点内存]
        V(mutex);

        // 归还内存资源
        for (int i = 0; i < size; i++) {
            V(memory_cells);
        }
    }
}

```

空间开销分析：该设计中，信号量的空间开销主要来自于三个信号量对象：mutex、items和memory_cells。每个信号量对象通常占用固定的内存空间。因此，总的空间开销为3倍单个信号量的大小。

9、分优先级写者问题:写者分为短写者（写操作耗时<10ms）、长写者（写操作耗时>100ms），要求短写者优先级高于长写者，但若长写者等待超过1s则优先级提升为最高优先执行；读者与所有写者互斥。请用信号量实现，分析是否会出现写者饿死。

信号量定义：

```

semaphore mutex = 1;
semaphore rw_mutex = 1;
semaphore short_ws = 0;
semaphore long_ws = 0;
semaphore readers_q = 0;

```

```

int sw_count = 0, lw_count = 0, r_count = 0;
int active_readers = 0;
bool busy = false;    // 保护临界区的状态

```

```

long lw_arrival_times[MAX_WAITING];
int lw_head = 0, lw_tail = 0;

```

```

void Exit_Region() {

    // 检查长写者是否等待超过 1s
    if (lw_count > 0 && (get_current_time() - lw_arrival_times[lw_head] > 1000)) {
        lw_count--;
        lw_head = (lw_head + 1) % MAX_WAITING;
        V(long_ws);
    }
    // 短写者优先
    else if (sw_count > 0) {
        sw_count--;
        V(short_ws);
    }
    // 普通长写者
    else if (lw_count > 0) {
        lw_count--;
        lw_head = (lw_head + 1) % MAX_WAITING;
        V(long_ws);
    }
    // 读者
    else if (r_count > 0) {
        busy = true;
        while (r_count > 0) {
            r_count--;
            active_readers++;
            V(readers_q);
        }
    }
    else {
        busy = false;
    }
}

```

```

}

```


短写者	长写者
<pre>void Short_Writer() { P(mutex); if (!busy) { busy = true; } else { sw_count++; P(short_ws); // 等待调度 } P(rw_mutex); // [写操作] V(rw_mutex); Exit_Region(); V(mutex); }</pre>	<pre>void Long_Writer() { P(mutex); long now = get_current_time(); if (!busy) { busy = true; } else { lw_count++; lw_arrival_times[lw_tail] = now; // 记录到达时间 lw_tail = (lw_tail + 1) % MAX_WAITING; P(long_ws); // 等待调度 } P(rw_mutex); // 申请物理写锁 // [写操作 > 100ms] V(rw_mutex); Exit_Region(); V(mutex); }</pre>

读者：

```
void Reader() {
    P(mutex);
    // 读者优先策略：如果有写者在写或在排队，读者必须去队列等待
    if (busy || sw_count > 0 || lw_count > 0) {
        r_count++;
        V(mutex);
        P(readers_q);
    } else {
        active_readers++;
        if (active_readers == 1) {
            busy = true;
            V(rw_mutex); // 第一人负责锁定物理锁
        }
        V(mutex);
    }

    // [读取操作中...]

    P(mutex);
    active_readers--;
    if (active_readers == 0) {
        V(rw_mutex); // 最后一人释放物理锁
        V(mutex);
        Exit_Region(); // 读者全走后，触发调度检查是否有写者在等
    } else {
        V(mutex);
    }
}
```

饥饿分析：在该样例中，虽然短写者优先于长写者，但由于长写者等待超过1秒会被提升为最高优先级，因此长写者反而不会饿死。假设有大量长写者持续到达且操作时都大于1s，那么长写者会频繁被提升优先级，导致短写者饿死。

10、AND信号量实现:用普通记录型信号量实现AND信号量的SP（同时申请多个信号量，要么全拿到要么全不拿）、SV（同时释放多个信号量）操作，要求无死锁，对比该实现与按顺序申请信号量方案的性能差异。

AND信号量实现：

```
semaphore mutex = 1;
semaphore S1, S2, ..., Sn = 1;
```

SP操作	SV操作
------	------

```
void SP(semaphore S[]) {
    bool success = false;
    int cnt = 0;
    int used = 0; //记录哪一个信号量被成功申请了
    while (!success) {
        P(mutex);

        if (S1.value() && S2.value() && ... && Sn.value()) {
            for (for int i = 0; i < S.length(); i++) {
                // 同时申请所有信号量
                if (P(s)) {
                    cnt ++;
                    used |= (1 << i); // 记录成功申请的信号量
                }
            }
            if (cnt == S.length()) success = true; // 全部申请成功
        } else {
            // 申请失败，释放已申请的信号量
            for (int i = 0; i < s.length; i++) {
                if (used & (1 << i)) {
                    V(S[i]); // 释放已申请的信号量
                }
            }
        }
    }

    cnt = 0; used = 0;

    V(mutex);
}

void SV(semaphore S[]) {
    P(mutex);
    for (semaphore s : S) {
        V(s); // 同时释放所有信号量
    }
    V(mutex);
}
```

性能差异分析：与按顺序申请信号量方案相比，AND信号量实现的SP操作可能会导致更多的等待和重试，因为它要求所有信号量必须同时可用才能成功申请。这可能会增加系统的响应时间，尤其是在高竞争的情况下。而按顺序申请信号量方案虽然可能存在死锁风险，但在资源较为充足时通常能更快地获得所需资源。即是说，AND信号量的并发性不如按顺序申请的方案，实时性也较差，CPU利用率可能更低。

11、打印店资源同步:打印店有2台打印机、3台扫描仪，打印任务仅需申请1台打印机，扫描任务仅需申请1台扫描仪，复合任务需先申请打印机再申请扫描仪，完成后释放资源。请用信号量实现，要求无死锁，计算最多可同时处理多少个任务；若复合任务可任意顺序申请打印机和扫描仪，是否会死锁?给出死锁示例。

信号量定义：

```
semaphore printers = 2, scanners = 3;
```

打印任务	扫描任务	复合任务
<pre>void Print_Task() { P(printers); // [打印中...] V(printers); }</pre>	<pre>void Scan_Task() { P(scanners); // [扫描中...] V(scanners); }</pre>	<pre>void Composite_Task() { P(printers); P(scanners); // [复合任务中...] V(scanners); V(printers); }</pre>

最大同时处理任务数：没有复合任务的情况下，最多可以同时处理2个打印任务和3个扫描任务，共5个任务；如果有复合任务存在，最坏情况下可能同时占用2台打印机和3台扫描仪，因此最多只能同时处理2个复合任务和一个扫描任务，共3个任务。

死锁分析：如果复合任务可以任意顺序申请打印机和扫描仪，若有两个复合任务同时执行，假设此时剩余一个打印机和扫描仪一个先申请了打印机A但还未申请扫描仪，另一个先申请了扫描仪B但还未申请打印机，则两个复合任务会相互等待对方释放资源，导致死锁。

12、内核态信号量实现:在单CPU抢占式内核中，用开关中断实现二元信号量的P/V操作，写出完整代码；说明该实现为什么在多核系统中不可行，给出多核场景下的改进方案。

```
typedef struct {
    int value;
    struct list_head wait_queue;
} binary_semaphore;
```

P操作	V操作
<pre>void P(binary_semaphore *sem) { unsigned long flags; local_irq_save(flags); while (sem->value == 0) { // 资源不可用，将当前进程加入等待队列 add_current_to_queue(&sem->wait_queue); // 将当前进程设为睡眠状态 set_current_state(TASK_INTERRUPTIBLE); // 重新开中断并触发调度 local_irq_restore(flags); schedule(); // 被唤醒后，重新关中断进入循环检查 local_irq_save(flags); } sem->value = 0; // 占用资源 local_irq_restore(flags); }</pre>	<pre>void V(binary_semaphore *sem) { unsigned long flags; local_irq_save(flags); sem->value = 1; // 如果队列不为空，唤醒等待的进程 if (!list_empty(&sem->wait_queue)) { wakeup_first_process(&sem->wait_queue); } local_irq_restore(flags); }</pre>

在多核场景下，该实现不可行的原因是：在多核系统中，多个CPU可能同时执行P/V操作，导致竞态条件和数据不一致问题。由于该实现依赖于单CPU环境下的中断控制来保证互斥，在多核环境中，其他CPU可能会同时访问和修改信号量的状态，导致死锁或资源泄漏。

修改方案：在多核系统中，可以使用自旋锁来保护信号量的状态，确保同一时间只有一个CPU能够修改信号量。

```
void P(binary_semaphore *sem) {
    spin_lock(&sem->lock);

    while (sem->value == 0) {
        add_current_to_queue(&sem->wait_queue);
        set_current_state(TASK_INTERRUPTIBLE);

        // 释放自旋锁并调度
        spin_unlock(&sem->lock);
        schedule();

        spin_lock(&sem->lock);
    }

    sem->value = 0;
    spin_unlock(&sem->lock);
}
```

13、过河同步问题:东西两岸各有若干人需要过河，仅1条船最多载4人，船必须有人驾驶才能移动，且船到岸后所有人必须下船，船在某岸时只有该岸的人可以上船。请用信号量实现两岸乘客、船的同步，要求不会出现船空等、某岸乘客饿死的情况。

信号量实现：

```
semaphore mutex = 1;
semaphore east_can_board = west_can_board = 0;
semaphore onboard_done = offboard_done = 0;
semaphore sailing = 0;

int east_wait = 0;
int west_wait = 0;
int current_onboard = 0;
int side = 0; // 0代表东岸，1代表西岸
```

乘客在两岸的行为是对称的，这里用东岸举例

```
void East_Passenger() {
    P(mutex);
    east_wait++;
    V(mutex);

    // 等待船到东岸且允许上船
    P(east_can_board);

    P(mutex);
    current_onboard++;
    if (current_onboard == 4 || (current_onboard >= 1 && east_wait == 0)) {
        // 船满了，或者虽然没满但东岸没人了
        V(onboard_done);
    } else {
        V(east_can_board); // 允许下一个东岸乘客上船
    }
    V(mutex);

    P(sailing); // 等待航行到达对岸

    // 下船
    P(mutex);
    current_onboard--;
    if (current_onboard == 0) {
        V(offboard_done); // 最后一人下船后通知船
    }
    V(mutex);
}
```

船

```
void Boat() {
    while (true) {
        // --- 在东岸准备载客 ---
        P(mutex);
        side = 0;
        if (east_wait > 0) {
            V(east_can_board); // 开启东岸上船闸门
            V(mutex);
            P(onboard_done);    // 等待上船结束
        } else {
            V(mutex);          // 东岸没人，直接准备去西岸（防止空等）
        }

        // 航行到西岸
        int saved_count = current_onboard;
        for(int i=0; i<saved_count; i++) V(sailing); // 释放到达信号
        if (saved_count > 0) P(offboard_done);      // 等待所有人下船

        // --- 在西岸准备载客 ---
        P(mutex);
        side = 1;
        if (west_wait > 0) {
            V(west_can_board); // 开启西岸上船闸门
            V(mutex);
            P(onboard_done);    // 等待上船结束
        } else {
            V(mutex);
        }

        // 航行回东岸
        saved_count = current_onboard;
        for(int i=0; i<saved_count; i++) V(sailing);
        if (saved_count > 0) P(offboard_done);
    }
}
```

14、信号量性能分析:某临界区执行时间为 $1ms$ ，进程切换开销为 $0.1ms$ ，现有10个进程同时请求进入临界区。分别计算用忙等自旋信号量、阻塞挂起型信号量实现时的总耗时，判断哪种方案效率更高；若临界区执行时间改为 $10\mu s$ ，结果会发生什么变化？

忙等自旋信号量: $T = 10 \times 1ms = 10ms$

阻塞挂起型信号量: $T = 10 \times 1ms + 9 \times 0.1ms = 10.9ms$

在这种情况下，两种机制的效率差异不大。

如果临界区执行时间改为10μs，

忙等自旋信号量: $T = 10 \times 10\mu s = 0.1ms$

阻塞挂起型信号量: $T = 10 \times 10\mu s + 9 \times 0.1ms = 1ms$

当进程切换开销远大于临界区执行时间时，忙等自旋信号量的效率会显著高于阻塞挂起型信号量。

15、管程与信号量转换:用信号量实现管程的三个核心逻辑:互斥访问、条件变量wait操作、条件变量signal操作，说明为什么管程比原生信号量更容易避免死锁。

信号量定义：

```
semaphore mutex = 1;
semaphore next = 0;

int next_count = 0;

semaphore x_sem = 0;
int x_count = 0;
```

互斥访问	条件变量wait	条件变量signal
<pre>... P(mutex); // [临界区] if (next_count > 0) { V(next); } else { V(mutex); } ...</pre>	<pre>void wait(x) { x_count++; if (next_count > 0) // 唤醒之前因为 signal 而挂起的进程 V(next); else V(mutex); P(x_sem); x_count--; }</pre>	<pre>void signal(x) { if (x_count > 0) { next_count++; // 唤醒在条件变量上等待的进程 V(x_sem); P(next); next_count--; } }</pre>

管程一次只允许一个进程调度上述操作，保证了互斥性；
同时使用了条件变量来实现同步。

16、左右手哲学家问题:6个哲学家围坐，共6根筷子，其中3个是左撇子（先拿左手边筷子）、3个是右撇子（先拿右手边筷子）。请设计座位排布规则避免死锁，用信号量实现该规则并证明无死锁。

由于已经限定了左右撇子的数量和行为，所以可以证明，在题目给定的条件下，不会产生死锁。

如果会发生死锁，那么每个哲学家第一次选择的筷子方向必须都相同，即构成了一个环。
由于已经限定了左右撇子的数量和行为，所以不可能出现所有哲学家都选择同一方向的情况，因此，环不会出现，死锁也就不会发生。

座位排布方式： 两个左撇子中间坐着的必须是右撇子，两个右撇子中间坐着的必须是左撇子。

实现：

```
// 奇数位坐着右撇子，偶数位坐着左撇子
semaphore chopsticks[6] = {1};
void philosopher(int i) {
    while (true) {
        if (i % 2 == 0) {
            P(chopsticks[i]);
            P(chopsticks[(i + 1) % 6]);
        } else {
            P(chopsticks[(i + 1) % 6]);
            P(chopsticks[i]);
        }
    }

    // [吃饭中...]

    V(chopsticks[i]);
    V(chopsticks[(i + 1) % 6]);
}
---
```

17、多类型消费者问题:两个生产者P1/P2分别生产D1/D2两类数据，三个消费者C1仅消费D1、C2仅消费D2、C3每次必须同时消费1个D1和1个D2的组合，缓冲区最多存放5个D1和5个D2。请用信号量实现同步互斥，要求C3不会饿死，定义所有信号量的含义与初值。

由于不想处理关于提高C3优先级的复杂逻辑，本解法采用以下逻辑：
在资源消耗阶段，将 C1, C2, C3 相同的对象，即他们都会申请消耗 D1 和 D2 的信号量，只有当两个信号量都申请到时才会进入临界区消耗资源，否则就把资源归还，重新申请。
在消耗时，C1 和 C2 不会真正消耗他们不需要的资源，而是在消耗了对应的资源后，立刻将不需要的资源归还到缓冲区中。
具体实现如下：

信号量定义

```
semaphore mutex = 1;

semaphore empty_d1 = empty_d2 = 5;

semaphore full_d1 = full_d2 = 0;

int d1_count = 0, d2_count = 0;
```

生产者P1	生产者P2
<pre>void P1() { while(true) { P(empty_d1); P(mutex); // [生产 D1...] d1_count++; V(mutex); V(full_d1); } }</pre>	<pre>void P2() { P(empty_d2); P(mutex); // [生产 D2...] d2_count++; V(mutex); V(full_d2); } }</pre>

消费者C1	消费者C2	消费者C3
<pre>void C1() { while (true) { P(mutex); if (d1_count > 0 && d2_count > 0) { P(full_d1); P(full_d2); // [消耗 D1] d1_count--; V(empty_d1); V(full_d2); V(mutex); } else if (d1_count > 0 && d2_count == 0) { V(mutex); } else { V(mutex); } } }</pre>	<pre>void C2() { while (true) { P(mutex); if (d2_count > 0 && d1_count > 0) { P(full_d1); P(full_d2); // [消耗 D2] d2_count--; V(empty_d2); V(full_d1); V(mutex); } else if (d2_count > 0 && d1_count == 0) { V(mutex); } else { V(mutex); } } }</pre>	<pre>void C2() { while (true) { P(mutex); if (d2_count > 0 && d1_count > 0) { P(full_d1); P(full_d2); // [消耗 D1 和 D2] d1_count--; d2_count--; V(empty_d2); V(empty_d1); V(mutex); } else { V(mutex); } } }</pre>

18、机场值机同步问题:机场有10个值机柜台，每个柜台同一时间仅服务1个乘客；乘客分为国内、国际两类，国际乘客需要额外护照检查，共有2个检查岗，值机柜台服务国际乘客时必须先申请1个检查岗，服务完成后释放，国内乘客无需检查。请用信号量实现同步，要求国际乘客平均等待时间不超过国内乘客的2倍，避免饿死。

信号量定义：

```
semaphore counters = 10;
semaphore check_point = 2;
semaphore mutex = 1;

semaphore domestic_queue = 0;
semaphore internat_queue = 0;

int dom_wait_count = 0;
int int_wait_count = 0;
int counters_busy = 0;
int int_served_streak = 0;
int dom_served_streak = 0;

const int MAX_DOM_STREAK = 2; //一个柜台连续服务国内乘客的最大次数
const int MAX_INT_STREAK = 1;
```

值机柜台：

```
void Counter_Service() {
    while (true) {
        P(mutex);

        bool can_serve_int = (int_wait_count > 0 && check_point.value > 0);
        bool can_serve_dom = (dom_wait_count > 0);

        int target = -1;

        if (can_serve_int && can_serve_dom) {
            if (int_served_streak < MAX_INT_STREAK) target = 1;
            else target = 0;
        }
        else if (can_serve_int) target = 1;
        else if (can_serve_dom) target = 0;

        if (target == 1) { // 国际乘客
            int_wait_count--;
            int_served_streak++;
            dom_served_streak = 0;
            P(check_point);
            V(internat_queue);
        } else if (target == 0) { // 国内乘客
            dom_wait_count--;
            dom_served_streak++;
            int_served_streak = 0;
            V(domestic_queue);
        } else {
            // 没人或国际乘客缺检查岗，柜台进入休眠或处理其他事务
        }

        V(mutex);

        // [执行服务中...]
    }
}
```

国内乘客	国际乘客
<pre>void Domestic_Passenger() { P(mutex); dom_wait_count++; V(mutex); P(domestic_queue); // 等待 // [正在值机...] }</pre>	<pre>void International_Passenger() { P(mutex); int_wait_count++; V(mutex); P(internat_queue); // 等待 // [值机 + 护照检查] V(check_point); }</pre>

19、分区读写同步问题:共享数据分为3个独立块B1/B2/B3，'写者W1仅修改B1、W2仅修改B2、W3仅修改B3，读者R1读B1+B2、R2读B2+B3、R3读B1+B3；'写者修改某块时仅与读该块的读者互斥，不同块的写者可并发执行。请用信号量实现同步，计算该机制的最大并

发度。

信号量定义：

```
semaphore mutex_B1 = 1, mutex_B2 = 1, mutex_B3 = 1;
smeaphore w_B1 = 1, w_B2 = 1, w_B3 = 1;

int rc1, rc2, rc3 = 0; // 读者计数器
```

写者W1(W2,W3相同逻辑)	读者R1(R2,R3相同逻辑)
<pre>void W1() { P(w_B1); // [修改 B1...] V(w_B1); }</pre>	<pre>void R1() { // 读 B1 P(mutex_B1); rc1++; if (rc1 == 1) P(w_B1); // 锁定 B1 // [读取B1] rc1--; if (rc1 == 0) V(w_B1); V(mutex_B1); // 读 B2 P(mutex_B2); rc2++; if (rc2 == 1) P(w_B2); // [读取B2] rc2--; if (rc2 == 0) V(w_B2); V(mutex_B2); }</pre>

最大并发度为 3，即 W1, W2, W3 同时执行。

20、实现一个写者绝对优先的读者-写者模型，读者可能饿死，分析其适用性。如果允许最多K个读者同时读，但写者无需等待所有读者完成，如何设计？

信号量定义：

```
semaphore mutex = 1;
semaphore r_sem = w_sem = 0;
semaphore rw_mutex = 1;

int r_count = w_count = 0;
int active_readers = 0;
bool writing = false;

void Exit_Region() {
    P(mutex);
    if (w_count > 0) {
        w_count--;
        writing = true;
        V(w_sem);
    } else if (r_count > 0) {
        writing = false;
        while (r_count > 0) {
            active_readers++;
            r_count--;
            V(r_sem);
        }
    } else {
        writing = false;
    }
    V(mutex);
}
```

写者	读者
----	----


```
void Writer() {
    P(mutex);
    if (!writing && active_readers == 0) {
        writing = true;
        V(mutex);
    } else {
        w_count++;
        V(mutex);
        P(w_sem);
    }

    P(rw_mutex);
    // [写操作]
    V(rw_mutex);

    Exit_Region();
}
```

```
void Reader() {
    P(mutex);
    if (writing || w_count > 0) {
        r_count++;
        V(mutex);
        P(r_sem);
    } else {
        active_readers++;
        if (active_readers == 1) P(rw_mutex);
        V(mutex);
    }

    // [读操作]

    P(mutex);
    active_readers--;
    if (active_readers == 0) {
        V(rw_mutex);
        Exit_Region();
    } else {
        V(mutex);
    }
}
```

如果允许最多K个读者同时读，但写者无需等待所有读者完成，如何设计？

```
semaphore k_limit = K;
semaphore mutex = 1;
semaphore update_lock = 1;

int reader_count = 0;
int active_idx = 0;
```

写者	读者
<pre>void Reader() { P(k_limit); P(mutex); reader_count++; V(mutex); //[读取] P(mutex); reader_count--; V(mutex); V(k_limit); }</pre>	<pre>void Writer() { P(update_lock); int next_idx = 1 - active_idx; //[写入] P(mutex); active_idx = next_idx; V(mutex); V(update_lock); }</pre>

21、快递驿站同步建模:快递驿站有3个入库台、5个取件台、2个生鲜冷藏柜。角色包括:快递员（需要占用入库台存件，生鲜件必须存入冷藏柜）、取件用户（普通件占用取件台取件，生鲜件需要先到冷藏柜取件再到取件台核验）、巡检员（每小时需要独占所有冷藏柜盘点

10分钟)。要求生鲜件取件优先级高于普通件，快递员入库优先级低于用户取件。请实现同步逻辑，分析高峰期取件用户的队列长度上限。

```
Monitor ExpressStation {
    int input_stages = 3;
    int output_stages = 5;
    int cold_lockers = 2;

    int fresh_wait_count = user_wait_count = 0;
    bool inspector_working = false;
    int lockers_in_use = 0;

    Condition can_input;
    Condition can_pick_normal, can_pick_fresh;
    Condition can_inspect;

    void signal_next() {
        if (lockers_in_use == 0) can_inspect.signal();
        if (!inspector_working && cold_lockers > 0 && output_stages > 0)
            can_pick_fresh.signal();
        else if (output_stages > 0)
            can_pick_normal.signal();
        else if (input_stages > 0)
            can_input.signal();
    }

    void input_package(bool is_fresh) {
        while (input_stages == 0 ||
            (is_fresh && (cold_lockers == 0 || inspector_working)) ||
            user_wait_count > 0 ||
            fresh_wait_count > 0) {
            can_input.wait();
        }
        input_stages--;
        if (is_fresh) {
            cold_lockers--;
            lockers_in_use++;
        }
        // [存]
        input_stages++;
        if (is_fresh) {
            cold_lockers++;
            lockers_in_use--;
        }
        signal_next();
    }

    void pick_package(bool is_fresh) {
        if (is_fresh) {
            fresh_wait_count++;
            while (output_stages == 0 || cold_lockers == 0 || inspector_working) {
                can_pick_fresh.wait();
            }
            fresh_wait_count--;
            cold_lockers--;
            lockers_in_use++;
            output_stages--;
            // [取件并核验]
            cold_lockers++;
            lockers_in_use--;
            output_stages++;
        } else {
            user_wait_count++;
            while (output_stages == 0 || fresh_wait_count > 0) {
                can_pick_normal.wait();
            }
            user_wait_count--;
            output_stages--;
            // [取件]
            output_stages++;
        }
        signal_next();
    }

    void inspect() {
        while (lockers_in_use > 0) {
```

```

        can_inspect.wait();
    }
    inspector_working = true;
    // [ 盘点10分钟]
    inspector_working = false;
    signal_next();
}
}

```

22、RT公交同步建模:BRT线路有10个站点，每辆公交车核载60人，每个站点有2个上车闸机、1个下车闸机，换乘乘客可从换乘通道直接进入上车队列，优先级高于普通候车乘客。要求:车辆到站后先下后上，满员后不得上车，所有乘客上下完成后司机才能关门驶离。请实现司机、上车乘客、下车乘客、站台工作人员的同步逻辑，分析高峰期最小发车间隔。

```

Monitor BRT_Station {
    int up_gates = 2;
    int down_gates = 1;

    Condition gate_up_ready;
    Condition gate_down_ready;
    Condition trans_passenger_on;
    Condition normal_passenger_on;

    void bus_process() {
        // 先下车 等待空下车闸机
        while (down_gates == 0) gate_down_ready.wait();
        down_gates--;

        // [乘客下车...]

        down_gates++;
        gate_down_ready.signal();

        // 上车 等待空上车闸机
        while (up_gates == 0) gate_up_ready.wait();
        up_gates--;

        // 开始上车
        // 假设每个公交车对象传入了要上车的换乘乘客数量trans_num和普通乘客数量normal_num

        while (trans_num > 0 && num < 60) {
            trans_passenger_on.wait(); // 等待换乘乘客
            trans_num--; // 上车一个换乘乘客
            trans_passenger_on.signal(); // 允许下一个换乘乘客上车
        }
        while (normal_num > 0 && num < 60) {
            normal_passenger_on.wait(); // 等待普通乘客
            normal_num--; // 上车一个普通乘客
            normal_passenger_on.signal(); // 允许下一个普通乘客上车
        }

        up_gates++;
        gate_up_ready.signal();
    }
}

```

高峰期最小发车间隔分析:假设每个乘客上下车平均需要 t 秒，那么下车时间为 $60t$ ，上车时间为 $30t$ (均为最坏情况，即每辆车在每个站点都满载且更换一批乘客)。由于上车闸机的吞吐量大于下车闸机，因此最小发车间隔主要受下车时间限制，即最小发车间隔约为 $60t$ 秒。